Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin konu programı ve kaynak kitaplar.
- Algoritma kavramı ve iyi bir algoritmanın özellikleri: doğruluk (correctness), anlaşılabilirlik, kolay gerçeklenme, değiştirilebilirlik.
- Zaman verimliliği (time efficiency) ve yer verimliliği (space efficiency).
- Algoritmayı ifade etme yolları: akış diyagramı (flowchart), sözde kod (pseudo code), programlama dili.
- Akış diyagramı kuralları: başlangıç/bitiş, işlem kutusu, `if`, `for` ve `while` döngüleri.
- C programlama kuralları.
- Temel veri yapılarına genel bakış: dizi (array), bağlı liste (linked list), yığın (stack), kuyruk (queue), ağaç (tree), graf (graph), hash.
- Bubble sort ve erken duran (exchange bayraklı) sürümü.
- Selection sort.
- Zaman ve yer karmaşıklığı, en iyi/ortalama/en kötü durum (best/average/worst case).
- Sırasız ve sıralı dizide doğrusal arama (linear search).
- Insertion sort'a kısa giriş (ayrıntısı sonraki derste).

Hocanın Özellikle Vurguladığı Kısımlar

- Bir problemi çözerken birinci öncelik kodlamak değil, algoritmayı tasarlamaktır.
- Akış diyagramında tek bir başlangıç ve tek bir bitiş noktası olur. Sembol hataları en sık yapılan hatalardan biri.
- `if` kutusunun true kolu hiçbir zaman boş, false kolu dolu bırakılmaz. Gerekirse koşul ters çevrilir.
- True/false kolunun hangi tarafta olduğu önemli değil, ama hangisi olduğu mutlaka yazılmalı.
- Adım sayısı belliyse `for`, belli değilse (koşula bağlıysa) `while` kullanılır.
- `for` döngüsünü `break` ile ya da döngü değişkenine sınır dışı değer atayarak bitirmeyin. Böyle bir çıkış gerekiyorsa `while` kullanın.
- `for`'un bitiş kısmına koşul yazılmaz, bir değişken ya da sabit yazılır. Örneğin önce `n = x * y` hesaplanır, sonra `n` kullanılır.
- Koşulları birleştirirken AND ile OR sık karıştırılıyor. "Her iki şart sağlandığı sürece" AND demektir.
- Swap işlemi en çok hata yapılan yerlerden. Mutlaka geçici bir değişkenle yapılmalı.
- Dizilerde sınır ihlali yapmayın. Dizinin dışındaki bir göze erişmek segmentation fault ile programı çökertebilir.
- Algoritma karşılaştırmasında önce zaman karmaşıklığına bakılır. Eşitse bellek kullanımı düşük olan seçilir. Bellek kısıtı varsa yer verimliliği öne geçebilir.
- Analizde esas alınan senaryo en kötü durumdur. En iyi durumun olasılığı düşük, ortalama durumun hesabı zordur.
- Sıralı mı sırasız mı veriyle çalıştığınız, tasarımda sormanız gereken ilk sorulardandır.
- Dersi zamanında takip etmek uzun vadede başarıyı artırır.

Kısa Tekrar Notları

- Program: algoritma analizi, özyineleme, liste ve bağlı liste, yığın ve kuyruk, ağaçlar, heap ve heap sort, öncelikli kuyruk, graflar, metin arama ve sıralama algoritmaları.
- Program: Algoritma + Veri Yapıları (Niklaus Wirth).
- Bubble sort: komşu elemanlar karşılaştırılır, her turda en büyük eleman sona gider. En kötü durum $O(n^2)$.
- Exchange bayraklı bubble sort: bir turda hiç yer değişmezse dizi sıralıdır, algoritma durur. Sıralı dizide tek turda biter.
- Selection sort: her turda kalan elemanların en küçüğü bulunur, tur sonunda tek bir swap yapılır. Her durumda $O(n^2)$ karşılaştırma, ama daha az swap.
- Sırasız dizide doğrusal arama: en kötü durumda n karşılaştırma, $O(n)$.
- Sıralı dizide doğrusal arama: aranan değerden büyük bir elemana gelince arama durdurulur.
- Insertion sort: iskambil kâğıdı dizer gibi, her eleman önündeki sıralı kısımda yerine yerleştirilir. Küçük veri gruplarında tercih edilir.

Detaylı Açıklamalar

Haftalık program

Hafta	Konu
1	Dersin tanıtımı, temel bilgilerin tekrarı
2	Algoritma analizi, temel notasyonlar
3	Özyinelemeli (recursive) algoritmalar
4	Listeler ve bağlı listeler
5	Kuyruk ve yığın
6-7	Ağaçlar, heap ağaçları ve heap sort
8	Öncelikli kuyruk (priority queue)
11-12	Graflar
13-14	Metin (string) arama algoritmaları, sıralama algoritmalarının karşılaştırması

- Hash veri yapısı bu dönem işlenmeyecek. Gelecek dönem Algoritma Analizi dersinde anlatılacak.

Kaynaklar

- Ana kitap önerisi: *Introduction to Algorithms* (MIT Press).
- Ek kaynaklar:
  - Levitin, *Introduction to the Design and Analysis of Algorithms*.
  - Skiena, *The Algorithm Design Manual*.
  - Sedgewick, *An Introduction to the Analysis of Algorithms*.

C programlama kuralları

- C kodu ANSI C standartlarına uygun yazılmalı ki kod her platformda çalışsın.
- Global değişkenlerden kaçının. Değerleri fonksiyonlara parametre olarak aktarın.
- Değişken ve fonksiyon adları anlamlı olmalı. `a`, `b`, `aa` gibi adlar kullanmayın.
- Modüler programlama yapın: problemi alt problemlere ayırıp her biri için fonksiyon yazın. Böylece kod yeniden kullanılabilir.
- Mümkün olduğunca dinamik bellek ayırın (dynamic memory allocation), gerektiği kadar yer kullanın.
- Gerçekten açıklama gereken yerlere açıklama satırı (comment) yazın.

Algoritma ve verimlilik

- Algoritma: verilen bir girdi için istenen çıktıyı sonlu zamanda üreten adımlar dizisi.

- Tasarım kriterleri:
  - Doğruluk: tüm senaryolarda doğru çıktıyı üretmek. Birinci öncelik budur.
  - Kolay gerçeklenme ve anlaşılabilirlik. Başkası kodu devraldığında kolayca devam edebilmeli.
  - Değiştirilebilirlik.
  - Probleme uygun dil seçimi. Gömülü sistemlerde C, veri işlerinde Python, olay güdümlü (event driven) programlamada Java/C++ gibi.
- Zaman karmaşıklığı (time complexity): algoritmanın işi yaklaşık kaç adımda bitirdiği.
- Yer karmaşıklığı (space complexity): algoritmanın girdi boyutuna bağlı olarak ne kadar bellek kullandığı. Ek dizi ya da matris kullanmak yer karmaşıklığını artırır.

Akış diyagramı kuralları

- Başlangıç (start) ve bitiş (stop): her algoritmada birer tane.
- İşlem kutusu: atama, dört işlem gibi işler. Bir kutuya birden fazla işlem yazılabilir.
- Karar kutusu (`if`): içine karşılaştırma yazılır (`<`, `>`, `<=`, `>=`, `==`, `!=`). İki kol çıkar: true ve false.
- `for` döngüsü: başlangıç değeri, bitiş değeri ve adım (artış ya da azalış) yazılır. Geriye doğru sayarken adım `-1` olur.
- `while` döngüsü: bir ya da birden fazla koşul sağlandığı sürece devam eder. Önce kontrol, sonra iş yapılır.
- `do-while` benzeri yapıda önce iş yapılır, sonra koşul kontrol edilir.

Bubble sort

- Örnek dizi: 3, 5, 1, 2, 7, 4.
- Komşu iki eleman karşılaştırılır. Sıradaki eleman küçükse yer değiştirilir.
- Birinci turun sonunda en büyük eleman (7) en sona gider. Sonraki turlarda ikinci, üçüncü en büyük eleman yerine oturur.
- Her turda küçük elemanlar da birer adım başa yaklaşır.
- Sonuç: 1, 2, 3, 4, 5, 7.

```
for (i = 0; i <= n - 2; i++)
    for (j = 0; j <= n - 2 - i; j++)
        if (a[j] > a[j + 1]) {
            tmp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = tmp;
        }
```

- İç döngü her turda bir eleman daha kısa. Çünkü sondaki elemanlar artık yerinde.
- Dezavantaj: çok fazla atama yapılıyor. Her atama bellek erişimi demek. Bellek erişimi CPU döngüsünden çok daha yavaş, veri önbellekte (cache) değilse bu ciddi zaman kaybı.
- İç içe iki döngü olduğu için en kötü durumda $O(n^2)$.

Exchange bayraklı bubble sort

- Dizi zaten sıralıysa (1, 2, 3, 4, 5, 7) klasik bubble sort yine bütün karşılaştırmaları yapar.
- İyileştirme: dış döngü `while` olur ve bir `exchange` değişkeni turda yer değiştirme olup olmadığını izler.

```
exchange = 1;
i = 0;
while (i < n - 1 && exchange == 1) {
    exchange = 0;
    for (j = 0; j < n - 1 - i; j++)
        if (a[j] > a[j + 1]) {
            tmp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = tmp;
            exchange = 1;
```

```
        }
    i++;
}
```

- Bir turda hiç yer değişmezse `exchange` 0 kalır ve döngü biter.
- Sıralı ya da sıralıya yakın dizilerde (örneğin 1, 3, 2, 4, 5, 7) çok daha erken durur. En kötü durum dışında her zaman daha avantajlıdır.

Selection sort

- Her turda kalan elemanlar arasında en küçüğü aranır. Yalnız en küçüğün indisi (`min`) saklanır.
- Tur bitince başlangıçtaki elemanla bulunan en küçük eleman bir swap ile yer değiştirir.
- Örnek: 3, 5, 1, 2, 7, 4. İlk turda 1 bulunur, 3 ile yer değiştirir. İkinci turda 5'in yerine 2 gelir.

```
for (i = 0; i <= n - 2; i++) {
    min = i;
    for (j = i + 1; j <= n - 1; j++)
        if (a[j] < a[min])
            min = j;
    tmp = a[i];
    a[i] = a[min];
    a[min] = tmp;
}
```

- Swap'ı iç döngüde her yeni küçük bulunduğunda yapmak da mümkün, ama verimsiz olur.
- Avantajı: bubble sort'a göre çok daha az swap, yani daha az bellek erişimi.
- Dezavantajı: dizi sıralı verilse bile $O(n^2)$ karşılaştırma yapar. Sıralıya yakın dizilerde bayraklı bubble sort daha hızlıdır.

Doğrusal arama

- Arama; ekleme, güncelleme ve silme işlemlerinin ön adımı olduğu için sıralama kadar önemli.
- Sırasız dizi: elemanlar tek tek aranan değerle karşılaştırılır. En kötü durumda eleman sondadır ya da hiç yoktur, n karşılaştırma yapılır, $O(n)$.
- Sıralı dizi (1, 2, 3, 5, 7, 8 içinde 6 aranıyor): 7'ye gelince aranan değerden büyük bir elemana ulaşılmıştır. Sonraki elemanların 6 olma ihtimali yoktur, arama durdurulur.
- Derste gösterilen akış diyagramı sıralı dizi içindir. İki koşul birlikte kontrol edilir: indis sınırı ve `x > a[i]`.

```
i = 0;
while (i < n - 1 && x > a[i])
    i++;
if (a[i] == x)
    printf("found");
else
    printf("not found");
```

- Sınır kontrolü kritik. Diziniz 0-5 indisliyse `a[6]`'ya bakmak size ait olmayan belleğe erişmektir. Bazen sorun çıkmaz, ama o bellek başka bir program tarafından kullanılıyorsa segmentation fault oluşur.

Insertion sort (giriş)

- İskambil kâğıtlarını elde sıralamaya benzer. Sırasız gelen kâğıt, önceki kâğıtlar arasında doğru yerine yerleştirilir.
- Örnek: 5, 1, 4, 3, 2, 7, 6.
  - 1, 5'in önüne geçer: 1, 5.
  - 4, 1 ile 5 arasına girer: 1, 4, 5.
  - 3, 1 ile 4 arasına girer: 1, 3, 4, 5.

  - 2, 1 ile 3 arasına girer.
- Küçük veri gruplarında verimlidir. Karmaşıklığı sonraki derste incelenecek.

Ders sonrası notlar

- Hoca bubble sort karmaşıklığını anlatırken iki döngünün sınırlarını ($n-2$ ve $n-2-i$) çarparak n^2'ye ulaştı. Bu yalnız kaba bir üst sınırdır. İç döngü her turda bir kısaldığı için toplam karşılaştırma sayısı $\sum_{i=0}^{n-2}(n-1-i) = \frac{n(n-1)}{2}$ olur. Sonuç yine $O(n^2)$'dir.
- Hoca Sedgewick'in *An Introduction to the Analysis of Algorithms* kitabının Java ağırlıklı olduğunu söyledi. Büyük olasılıkla Sedgewick ve Wayne'in Java ağırlıklı *Algorithms* kitabıyla karıştı. *An Introduction to the Analysis of Algorithms* (Sedgewick ve Flajolet) büyük olasılıkla matematiksel analize odaklanır.
- Sıralı dizide doğrusal arama kodunda döngü `i < n - 1` ile sınırlandığında son eleman da çıkıştaki `a[i] == x` kontrolüyle yakalanır. Döngü `i < n` ile yazılırsa, çıkışta `a[n]` okunarak sınır ihlali yapılabilir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.